WASP Software Engineering and Cloud Computing 2026
Software Engineering Module Assignment

Fahmina Kabir

September, 2026

1 Introduction

My research focus is AI and ML based intrusion detection systems for cyber physical systems and robot systems, with cybersecurity as the central concern. I look at two properties that matter for real deployment but are often overlooked in ML security papers, generalization across different hardware platforms, and robustness against both adversarial attacks and the limits of constrained hardware. I have studied this in two settings, unmanned aerial vehicles during my master's, and ROS2 based robotic platforms in my ongoing PhD work. Both settings rely on wireless and networked communication, which opens them up to attacks such as GPS spoofing, jamming, and denial of service. Since failures here can have real safety consequences, an IDS cannot be judged on offline accuracy alone. It needs to be evaluated as something that will run in the field, under distribution shift, adversarial pressure, and hardware constraints, with deployment costs such as latency and memory reported alongside detection performance. This essay uses that idea, that trustworthy deployment depends on more than model accuracy, as a thread connecting the course lectures, guest lectures, and assigned reading to my own research.

2 Lecture Principles

2.1 Verification and validation

I chose verification and validation as the first principle from the lectures. Verification asks whether a system has been built correctly, whereas validation asks whether it fulfils its intended real-world purpose. In machine learning, the data-processing pipeline, labels, data splits, implementation, and metric calculations can be verified. However, a model learns its behaviour from data rather than following a complete written specification. ML testing therefore places particular emphasis on validation: whether the model suits the actual problem, rather than only whether the experiment was implemented correctly.

This distinction shapes how we evaluate ML-based intrusion detection for resource-constrained cyber-physical platforms. Strong overall accuracy, precision, recall, or F1 scores are not sufficient evidence that a system is ready for deployment. Aggregate measures can hide serious weaknesses, for example, a detector may perform well on common traffic while missing a rare but security-critical attack. In my current project, we examine the performance for individual attack types along with overall results. This detailed analysis exposed limitations that aggregate metrics alone would not reveal. We also assessed inference time and model requirements, since a IDS that performs well offline is not useful if it cannot meet the platform's latency and resource limits. In another previous work focusing on generalized IDS for UAM environment, we evaluated not only the system's overall performance but also cross-UAV data analysis for better generalization, and also tried to handle the adversarial attack on the ML-based IDS to make it more robust. In my opinion, these analyzes made our evaluation process stronger, but still preliminary

validation evidence. Real on-device testing under realistic operating and adversarial conditions is still required to establish whether the system is fit for use.

2.2 From model to system: research-engineering loops

I chose the distinction between science and engineering as the second principle from the lectures. Science asks why something works and aims at explanation and discovery, while engineering asks how to make something work reliably under real constraints. Machine learning sits between the two: it uses scientific methods such as hypotheses and statistical evaluation, but it must also result in a working, maintainable system, which is an engineering concern. In practice, this produces a repeating loop of experimenting, deploying, observing what fails, and refining, rather than a process that ends once a good result is obtained. This can be described as a progression of maturity stages, moving from early experiments that simply check whether an idea works, through proof of principle and prototype development, to proof of concept in realistic conditions, integration into a larger system, mission-ready validation, and finally deployment with ongoing monitoring and maintenance. Although the stages are usually presented in order, real projects rarely move through them in a straight line. A requirement discovered late, such as needing the system's decisions to be explainable to an operator, can send development back to an earlier stage to rework the model itself, since the requirement should ideally have been identified before deployment concerns were addressed. A strong result at an early stage is therefore evidence that an idea might survive the later, harder stages, but it is not proof of engineering maturity by itself.

This distinction reframes how I think about ML-based intrusion detection for resource-constrained cyber-physical platforms. A model that performs well in an experiment answers a research question, such as whether the method can recognize attacks at all, but deploying it raises separate engineering questions, such as whether it can run dependably within a CPS security setup, meet latency and resource constraints, and remain observable and updatable after deployment. My published work in this area offers solid proof of principle and proof of concept style evidence, showing that a lightweight detection approach can generalize across different UAV platforms and withstand some adversarial pressure. This is valuable, but it does not yet demonstrate the later stages of integration, mission-ready validation, or field monitoring, and any gap found at those later stages, for example around explainability or operational reliability, would likely require revisiting earlier design choices rather than only adding a fix on top. My ongoing work on lightweight intrusion detection for resource-constrained ROS2 systems aims to move further along this progression, though it remains at an early stage. In my opinion, lightweight methods matter not only because they reduce computational cost, but because a smaller and simpler model is easier to monitor, retrain, and maintain on constrained hardware, which makes the later engineering stages more achievable. Even so, being lightweight does not remove the need for full system-level validation, since efficiency and validation remain separate concerns that both need to be addressed before such a system can be considered ready for real deployment.

3 Guest-Lecture Principles

3.1 Requirements Engineering for Security and Deployment Needs

I chose requirements engineering as the guest lecture principle. A key distinction from this lecture is the separation between three layers of abstraction: the context layer, which asks why the system exists, the requirements layer, which asks what the system must achieve, and the solution layer, which asks how it will achieve it. Using a lightweight ML model is not itself a requirement, it is one possible solution sitting in the solution layer. The actual requirement it serves might be something like detecting security relevant activity within an acceptable delay. This distinction matters because a solution can quietly become treated as the goal itself, while the real underlying need gets lost along the way.

Applied to an intrusion detection system running on a resource-constrained UAV or ROS2 platform, several requirement types deserve particular attention: security effectiveness, meaning the system detects attacks that actually matter to the mission rather than only ones that are easy to simulate; reliability, meaning consistent behaviour across the conditions the system will actually face; latency, meaning detections arrive fast enough for a response to still be possible; resource efficiency, meaning the system fits within the platform's compute and memory budget; false alarm control, since too many false alarms overwhelm operators or trigger unnecessary responses; maintainability and retrainability, so the system can be updated as attacks, platforms, or data change over time; and traceability, so that system goals, data, models, thresholds, and evaluation evidence can all be connected back to each other. In my opinion, requirements should drive the choice of model rather than the other way around. If a model performs strongly on a benchmark but cannot meet the latency, resource, or alert quality requirements of the actual platform, it is not the right engineering choice for that system, regardless of how good its accuracy looks in isolation.

3.2 Human Oversight, Monitoring, and Escalation

I chose human oversight as the second guest lecture principle. This lecture framed software and AI work as a socio-technical process rather than only a technical pipeline, one that includes reviews and approvals, quality assurance gates, automated evaluation, confidence thresholds, monitoring, escalation, and clear decision rights and accountability. An intrusion detection system does not simply output a label and stop there, that output enters a larger response process involving operators, security teams, maintainers, or in some cases automated countermeasures, so how well that process works matters just as much as how accurate the model is.

Once this human layer is taken seriously, several design questions become important: who receives an alert, since someone has to actually own the response or the alert goes nowhere; what information accompanies it, since operators need enough context to judge urgency; when a response can be automated, since automatic blocking or isolation can itself cause harm if it triggers incorrectly; what a low confidence detection should mean, since it ought to lead to review or extra logging rather than silently doing nothing; what happens after a false positive, since repeated false alarms wear down trust and cause operators to start ignoring the system; what happens after a false negative, since a missed attack can have consequences far larger than an average performance metric would suggest; and who is allowed to change thresholds or retrain the model, which is a governance question as much as a technical one. In my opinion, the practical quality of an intrusion detection system depends just as much on the human response workflow around it as on its raw detection performance. Evaluating detection accuracy without also evaluating alert presentation, escalation paths, accountability, and operator workload gives an incomplete picture of whether the system actually works in practice.

4 Data Scientists, Software Engineers, and AI Engineers

What differences do the chapters describe between data scientists and software engineers? Do you agree with this distinction? Why or why not?

The chapters describe data scientists as people whose core work is turning data into an accurate model, through data preparation, feature engineering, and hyperparameter tuning, evaluated mainly by accuracy on held out data. Software engineers instead focus on building a working product under real constraints, covering requirements, architecture, testing, deployment, and maintenance, and they are used to balancing several qualities at once rather than optimizing one metric.

I agree with the distinction, but with one addition. The chapters argue that few challenges of building production systems around a model are uniquely new, and that a lot of existing software engineering

knowledge already applies once it is pointed at ML. This suggests that what a software engineer usually lacks is not general problem solving skill but ML literacy, knowing that a model is an unreliable component and understanding enough about how it fails to apply existing engineering practices correctly. The reverse seems to hold too, the chapters note that data scientists without engineering training tend to write code that is poorly tested and hard to maintain. So the gap on both sides looks more like missing literacy about the other domain than a missing skill set, though the chapters also hint that training a model itself needs genuine specialized depth, not just literacy, so the two directions are not fully symmetric.

The chapters argue that production ML requires a system-wide view, not only a model-centric view. How does this apply to your own research area?

Chapter 2 argues that a model is only one part of a larger system, and judging it by accuracy alone misses whether the full system actually works. This fits intrusion detection for cyber physical systems directly. A detector is not the end goal, it supports a security response process with alerts, thresholds, and operators. A model with strong benchmark accuracy says little about whether it will actually protect a system or a platform once deployed, since that depends on latency, hardware limits, and how the detector behaves once embedded in the full pipeline rather than tested alone. My own work on generalization and adversarial robustness moves toward this system view by testing beyond a single dataset, but by the book's own framing this is still closer to proof of concept than full system level validation, since it does not yet cover monitoring or human response.

Do you think data scientists and software engineers will remain distinct roles, or will both need to learn more of each other's skills? Explain.

I do not think the two roles will stay fully separate. The chapters show the same pattern from both directions, software engineers without ML training build naive models, and data scientists without engineering training write code that is hard to maintain. This points to a lack of literacy about the other side rather than a lack of general skill, which suggests both roles will keep absorbing more of each other's basics over time. Full convergence into one role seems unlikely though, since the chapters treat model building as needing real specialized depth, not just surface familiarity. The more realistic path is what the book calls T shaped team members, deep in one area with working knowledge of the other.

Where does "AI Engineering" fit in this picture: is it mostly software engineering for AI systems, data science made more systematic, MLOps/model engineering, or something partly distinct?

AI engineering does not map cleanly onto software engineering for AI, systematic data science, or MLOps alone. It overlaps with all three and is most useful when it connects model level decisions to system level requirements directly. In my domain this means treating a lightweight detector's accuracy, latency, and resource use as requirements to satisfy together, rather than optimizing accuracy first and fitting the rest of the system around it later.

How do foundation models, AI coding assistants, RAG systems, prompt pipelines, and agentic workflows change the boundaries between these roles?

These developments add a layer that needs both statistical judgment and systems thinking at once, since a prompt pipeline behaves like a system of chained, unreliable components. For intrusion detection specifically, this shift matters less at the core detection task, since large foundation models are too slow and too large for a UAV or embedded ROS2 platform. It matters more at the tooling level, for example using a coding assistant to help build a detection pipeline, while the detector itself still needs a small, purpose built model rather than a general purpose one.

5 Paper Analysis

5.1 Paper 1: What About the Data? A Mapping Study on Data Engineering for AI Systems.

5.1.1 Core ideas and software-engineering importance

Heck reviews twenty five papers on data engineering for AI systems. They find that most research still treats data as a small step before modeling. It is rarely treated as its own engineering discipline. Only a few papers look at data engineering at the system or enterprise level. Most papers focus narrowly on a training pipeline or a production pipeline for one project. The paper's main contribution is showing that this narrow focus leaves practitioners without shared vocabulary, reference architectures, or best practices. This affects problems like data validation, quality checks, and lineage tracking. This matters for AI engineering because a model can only be as dependable as the pipeline that feeds it. Without an engineering discipline around that pipeline, claims about model performance rest on an unexamined foundation.

5.1.2 Relation to my research

Intrusion detection systems depends entirely on the features taken from sensors, logs, and network traffic. This happens before any model sees the data. A detector evaluated on a clean benchmark says little about whether the same pipeline stays trustworthy once deployed. Drift, missing values, or sensor faults in the pipeline can quietly lower detection quality. This can happen long before it shows up in an accuracy number. Heck's observation that most research stops at the training pipeline describes this gap well. It is the gap between a benchmarked classifier and a dependable intrusion detector.

5.1.3 Integration into a larger AI-intensive project

Suppose we consider a security platform managing a mixed fleet of UAVs and ROS2 based ground robots. Each unit runs a lightweight intrusion detector. A realistic architecture for this platform needs an enterprise level data layer. This layer would collect telemetry and security relevant features from every unit, check them against expected distributions, and serve validated, versioned feature sets to each onboard detector. Heck's review points toward this kind of layer directly. Several of the papers they survey describe similar data ecosystems for industrial settings, with defined roles for data engineers alongside data scientists and software engineers. My previous generalization and robustness work could sit inside this project as the modeling and evaluation component. It would consume feature streams that the data layer has already validated, rather than assuming clean input.

5.1.4 Adaptation of my research

A long term extension of my work could add an explicit data provenance and validation stage. This step would happen before any training or evaluation step. Anomalies in the input pipeline would then become a first class concern, rather than a silent assumption. This could include versioning datasets across platforms, logging where each feature came from, and running basic quality checks such as range and distribution tests before a model ever touches the data. This work fits under my ongoing research on lightweight intrusion detection for resource constrained ROS2 systems. Adding this layer would make generalization claims more credible, since they would rest on documented and checked data rather than a single static benchmark.

5.2 Paper 2: ROSPaCe: Intrusion Detection Dataset for a ROS2-Based Cyber-Physical System and IoT Networks.

5.2.1 Core ideas and software-engineering importance

ROSPaCe addresses a gap in evaluation engineering rather than in modeling. Many existing intrusion detection datasets cover cyber only systems. They monitor a single architectural layer, such as network traffic. They also generate attack and normal traffic in separate sessions. This prevents any detector trained on them from learning the realistic transition from a clean state to an under attack state. ROSPaCe collects data from three layers at once, the Linux operating system, the network, and the ROS2 middleware itself. It structures the dataset as repeating blocks of normal operation followed by a single attack. This allows measuring how long an attacker goes undetected, not only whether the detector separates classes correctly. The dataset also covers attacks specific to ROS2, not just generic network attacks, which is rare among existing datasets.

This shows that decisions about how data is collected and labeled directly shape what claims a later evaluation can support. That is itself a software engineering concern. A dataset built the wrong way can silently limit what any model trained on it will ever be able to demonstrate, no matter how good the model architecture is. In that sense, ROSPaCe is less about proposing a new detection method and more about fixing the evaluation infrastructure that detection research depends on.

5.2.2 Relation to my research

This paper sits close to the operational setting of my own research. ROS2 based cyber physical systems are the platform my ongoing work targets. The paper's emphasis on attack latency over raw accuracy matches a point worth stressing across my research generally. A security relevant detector should be judged on how quickly it catches an attacker and how many malicious actions occur first. It should not be judged only on how well it separates normal from attack samples in a shuffled test set. My past work on generalization across UAV platforms and adversarial robustness also depended on realistic evaluation setups, and ROSPaCe demonstrates one concrete way to build such a setup for a ROS2 based platform specifically, which is exactly the kind of resource constrained cyber physical system my ongoing research targets.

5.2.3 Integration into a larger AI-intensive project

Reusing the fleet security platform described above, this project would need exactly the kind of multi layer, block structured dataset that ROSPaCe provides. Such a dataset supports training detectors before deployment. It also supports continuous validation as new units, software versions, or attack types are added. This kind of dataset also supports monitoring and incident response. The alternating normal and attack blocks give a template for how live telemetry should be logged and labeled during an active investigation, since operators would already have a reference for what a real transition from normal to attack behavior should look like. My research on generalization across heterogeneous platforms fits naturally here. Fleet vehicles are rarely identical, and a detector trained on one unit's data needs to generalize to others without a fresh dataset for each one. A shared, well structured dataset like ROSPaCe would make it easier to test that generalization claim across many units rather than just one.

5.2.4 Adaptation of my research

A concrete improvement would be to restructure evaluation around explicit split policies that respect the block structure ROSPaCe uses. Training and test data should preserve realistic sequences rather than being

shuffled together. Future work should also report per attack class detection rates alongside latency, rather than a single aggregate score. Adding data quality checks and provenance tracking for each monitored layer would help too. A mechanism for detecting when deployment conditions have drifted from the training conditions would also help. Together these steps would push future work closer to a system that can be trusted in an operational fleet setting, rather than only in a benchmark. Adopting this kind of evaluation structure would also make it easier to compare future lightweight ROS2 detectors against each other fairly, since they would all be judged against the same realistic notion of attack latency rather than against differing, less transparent test setups.

6 Evidence, Originality, and GenAI-Use Transparency

I relied on four main sources: the course lecture material (Feldt, Frattini, and Lenberg), used to interpret the software-engineering principles I discussed; Chapters 1–2 of Kästner et al.'s online textbook *Machine Learning in Production*, used for the discussion of professional roles; Heck's peer-reviewed CAIN 2024 paper, used for the first paper analysis; and Puccetti et al.'s peer-reviewed *Scientific Data* paper, used for the second. I trust all of these sources, the two peer-reviewed papers for undergoing formal review and reporting concrete, reproducible methods, and the lecture material for drawing on and citing established, peer-reviewed research throughout. A GenAI tool was used for brainstorming, outlining, refining, and language checking. I read all sources myself, verified the claims, and wrote the analysis.

References

[1] R. Feldt, A. Frattini, and P. Lenberg. WASP Software Engineering Course Module lecture material, 2026.

[2] P. Heck. What About the Data? A Mapping Study on Data Engineering for AI Systems. In *Proceedings of the IEEE/ACM International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 2024.

[3] T. Puccetti, S. Nardi, C. Cinquilli, T. Zoppi, and A. Ceccarelli. ROSPaCe: Intrusion Detection Dataset for a ROS2-Based Cyber-Physical System and IoT Networks. *Scientific Data*, 11:481, 2024. `https://doi.org/10.1038/s41597-024-03311-2`

[4] C. Kästner, D. S. Arp, and collaborators. *Machine Learning in Production*, Chapters 1–2. `https://mlip-cmu.github.io/book/01-introduction.html`

[5] F. Kabir, N. I. Mowla, T. Rosenstatter, and I. Doh. GIIDS-AR: End-to-end generalized intelligent intrusion detection system with adversarial robustness for heterogeneous UAVs in UAM. *Computer Networks*, page 111821, 2025.